

50 Most Frequently Asked SQL Interview Questions

Follow and comment "PDF" to get the pdf.

Instagram

LinkedIn

WhatsApp Channel

Created by Shubham Soni

Swipe right →

Basic SELECT Queries

Foundational SELECT operations form the cornerstone of SQL proficiency. These queries demonstrate essential filtering, sorting, and pattern matching techniques that every developer must master.

Select All Records

SELECT * FROM employees;

Specific Columns

SELECT first_name, last_name, salary FROM employees;

WHERE Clause

SELECT * FROM employees WHERE salary > 50000;

DISTINCT Values

SELECT DISTINCT department_id FROM employees;

ORDER BY

SELECT * FROM employees ORDER BY salary DESC;

LIMIT Results

SELECT * FROM employees LIMIT 5;

LIKE Pattern

SELECT * FROM employees WHERE first_name LIKE 'J%';

IN Operator

SELECT * FROM employees WHERE department_id IN (10, 20, 30);

BETWEEN Range

SELECT * FROM employees WHERE salary BETWEEN 40000 AND 60000;

NULL Handling

SELECT * FROM employees WHERE manager_id IS NULL;

Aggregate Functions

Aggregate functions enable data summarization and statistical analysis. Understanding COUNT, SUM, AVG, MIN, MAX along with GROUP BY and HAVING clauses is critical for analyzing datasets and deriving meaningful insights.

1

COUNT Records

SELECT COUNT(*) FROM employees;

2

SUM Values

SELECT SUM(salary) AS total_salary FROM employees;

3

AVG Calculation

SELECT AVG(salary) AS avg_salary FROM employees;

4

MIN and MAX

SELECT MIN(salary), MAX(salary) FROM employees;

GROUP BY

SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;

HAVING Clause

SELECT department_id, AVG(salary) FROM employees GROUP BY department_id HAVING AVG(salary) > 50000;

Multiple Grouping

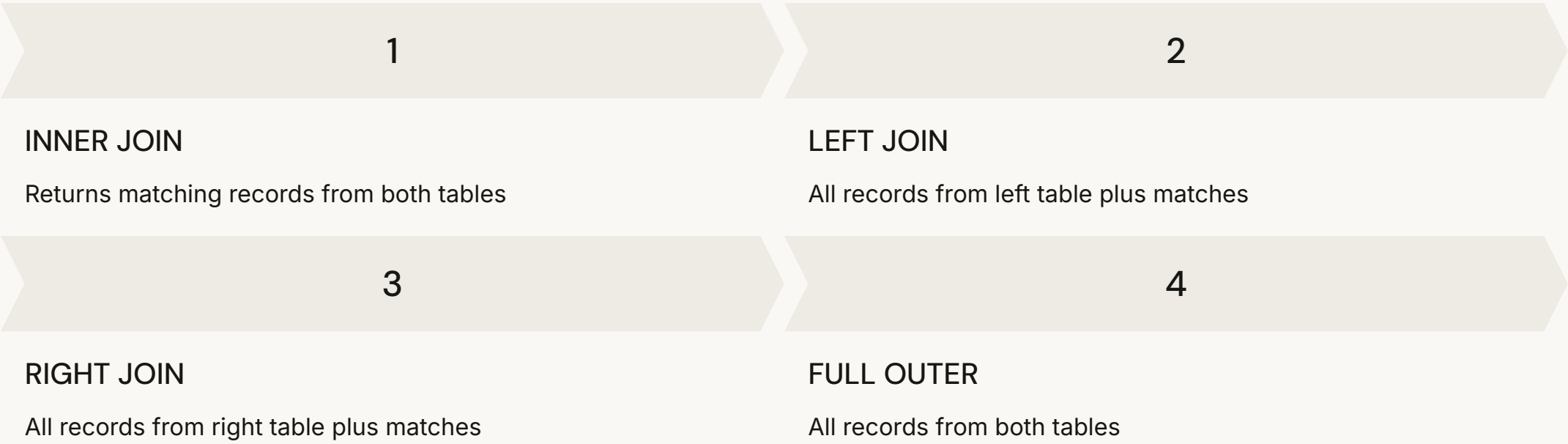
SELECT department_id, job_id, COUNT(*) FROM employees GROUP BY department_id, job_id;

Count Distinct

SELECT COUNT(DISTINCT department_id) FROM employees;

JOIN Operations

JOIN operations are fundamental for combining data from multiple tables. Mastering INNER, LEFT, RIGHT, FULL OUTER, and SELF joins allows you to retrieve related information and construct comprehensive datasets from normalized database structures.



Basic INNER JOIN

```
SELECT e.first_name, d.department_name FROM employees e
INNER JOIN departments d ON e.department_id =
d.department_id;
```

Self JOIN

```
SELECT e1.first_name AS employee, e2.first_name AS manager
FROM employees e1 LEFT JOIN employees e2 ON
e1.manager_id = e2.employee_id;
```

📌 **Multiple JOINS:** Chain multiple JOIN operations to combine data from three or more tables. Always consider performance implications and index usage when joining large datasets.

```
SELECT e.first_name, d.department_name, l.city FROM employees e JOIN departments d ON e.department_id = d.department_id JOIN
locations l ON d.location_id = l.location_id;
```

Subqueries

Subqueries enable complex filtering and nested logic within SQL statements. They can appear in WHERE, SELECT, or FROM clauses, providing powerful ways to break down complex problems into manageable steps.

O1	O2	O3
Subquery in WHERE	Subquery with IN	Correlated Subquery
Filter based on aggregated or derived values	Match against a set of returned values	Reference outer query in inner query
O4	O5	
EXISTS Operator	Derived Tables	
Check for existence of rows	Use subquery results as temporary tables	

Above Average Salary

```
SELECT * FROM employees WHERE salary > (SELECT
AVG(salary) FROM employees);
```

Departments with Employees

```
SELECT * FROM departments d WHERE EXISTS (SELECT 1 FROM
employees e WHERE e.department_id = d.department_id);
```

Created by Shubham Soni

Swipe right →

Advanced Queries

Advanced SQL techniques including window functions, CTEs, and complex analytical queries separate proficient developers from experts. These patterns handle sophisticated business logic and complex data analysis scenarios.

UNION Operations

```
SELECT first_name FROM
employees UNION
SELECT first_name FROM
managers;
```

CASE Statement

```
SELECT first_name, CASE
WHEN salary > 70000
THEN 'High' ELSE 'Low'
END FROM employees;
```

ROW_NUMBER()

```
SELECT first_name,
ROW_NUMBER() OVER
(ORDER BY salary DESC)
FROM employees;
```

PARTITION BY

```
SELECT first_name,
ROW_NUMBER() OVER
(PARTITION BY
department_id ORDER
BY salary DESC) FROM
employees;
```

Nth Highest Salary

```
SELECT DISTINCT salary FROM employees ORDER BY salary
DESC LIMIT 1 OFFSET 2;
```

Find Duplicates

```
SELECT email, COUNT(*) FROM employees GROUP BY email
HAVING COUNT(*) > 1;
```

📌 **Common Table Expressions (CTEs):** Use WITH clauses to create readable, reusable query components that improve maintainability and performance.

```
WITH dept_avg AS (SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY department_id) SELECT e.first_name,
e.salary FROM employees e JOIN dept_avg d ON e.department_id = d.department_id WHERE e.salary > d.avg_salary;
```

Created by Shubham Soni

Swipe right →

BONUS

Complex Interview Questions

These advanced scenarios test your ability to combine multiple SQL concepts. They frequently appear in senior-level interviews and require deep understanding of joins, window functions, and analytical thinking.

1	Employees Earning More Than Manager SELECT e.first_name AS employee, e.salary, m.first_name AS manager, m.salary FROM employees e JOIN employees m ON e.manager_id = m.employee_id WHERE e.salary > m.salary; Self-join to compare employee salaries with their managers
2	Second Highest Salary by Department SELECT department_id, salary FROM (SELECT department_id, salary, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rank FROM employees) ranked WHERE rank = 2; Combines window functions with filtering for departmental analysis
3	Running Total (Cumulative Sum) SELECT employee_id, salary, SUM(salary) OVER (ORDER BY employee_id) AS running_total FROM employees; Window function for calculating progressive totals across ordered rows

 **Interview Success Tips:** Practice writing queries by hand, explain your thought process aloud, consider edge cases, and always discuss performance implications. Understanding execution plans and indexing strategies demonstrates senior-level expertise.